# Compiler-Synthesized Holistic Attention (CSHA)
## A Novel Attention Architecture Uniquely Enabled by the NeuralScript Compiler

**Authors:** Brandon Wiemer
**Date:** April 2026
**Status:** Research Proposal

---

## Abstract

We propose **Compiler-Synthesized Holistic Attention (CSHA)**, an attention mechanism where the NSL compiler treats the entire transformer sublayer — from pre-norm through attention to residual connection — as a single optimizable unit, synthesizing a specialized GPU kernel that eliminates intermediate HBM round-trips that FlashAttention (even FA3) cannot avoid. CSHA further exploits NSL's compile-time weight analysis, roofline cost model, shape algebra, and source AD to generate per-layer specialized attention kernels with compile-time-determined sparsity patterns and per-head precision selection. Where FlashAttention optimizes the *inner* attention computation (QK^T → softmax → PV), CSHA optimizes the *outer* attention boundary — fusing the operations that surround attention into the attention kernel itself. This is fundamentally impossible in PyTorch because FlashAttention is an opaque pre-compiled CUDA library that the framework's compiler cannot see into or fuse across.

---

## 1. The HBM Boundary Problem

A standard pre-norm transformer block performs this sequence of operations:

```
Input x ∈ [B, S, D]                    ← Read from HBM
  → RMSNorm(x)                         ← Write to HBM, Read from HBM
  → Q = x_norm @ Wq                    ← Write Q to HBM
  → K = x_norm @ Wk                    ← Write K to HBM
  → V = x_norm @ Wv                    ← Write V to HBM
  → Q', K' = RoPE(Q, K)                ← Read Q,K from HBM, Write back
  → GQA reshape + expand               ← Read from HBM, Write back
  → FlashAttention(Q', K', V) → Attn_out   ← Read Q',K',V, Write Attn_out to HBM
  → Out = Attn_out @ Wo                ← Read Attn_out from HBM, Write Out
  → x + Out                            ← Read x and Out, Write result
```

**That is 10+ HBM round-trips** for what is logically a single transformation: `x → attention_sublayer(x)`.

FlashAttention (all versions) optimizes only the middle step: it fuses QK^T, softmax, and PV into a single tiled kernel that avoids materializing the S×S attention matrix. This was revolutionary — but it leaves the *boundary* operations untouched. Q, K, V must be fully materialized in HBM before FA reads them. FA's output must be written to HBM before the output projection reads it.

On an H100 with 3.35 TB/s HBM bandwidth and 989 TFLOPS FP16 compute, these boundary round-trips are *the* bottleneck. For the NSLCoder-50M at batch=1 seq=1024:

| Operation | FLOP | Bytes | AI (FLOP/byte) | Bound | Time (µs) |
|---|---|---|---|---|---|
| RMSNorm | 3M | 4MB | 0.75 | **Memory** | 1.2 |
| Q projection | 537M | 6MB | 89.5 | Compute | 0.5 |
| K projection | 269M | 4MB | 67.2 | Compute | 0.3 |
| V projection | 269M | 4MB | 67.2 | Compute | 0.3 |
| RoPE | 2M | 4MB | 0.5 | **Memory** | 1.2 |
| GQA reshape | 0 | 2MB | 0 | **Memory** | 0.6 |
| **FlashAttention** | 1070M | 6MB | 178 | Compute | 1.1 |
| Output projection | 537M | 6MB | 89.5 | Compute | 0.5 |
| Residual add | 0.5M | 4MB | 0.125 | **Memory** | 1.2 |
| **Total** | | **40MB** | | | **6.9** |

Of the 6.9µs total, approximately **4.2µs (61%) is spent on memory-bound boundary operations** that exist solely because the computation is split into separate kernels. The actual compute (projections + attention) takes only 2.7µs.

### Why PyTorch Cannot Fix This

PyTorch's `torch.compile` can fuse *elementwise* operations (RMSNorm, RoPE, residual add) into surrounding kernels. But it cannot fuse *across matmul boundaries* because:

1. FlashAttention is a pre-compiled CUDA library linked via ctypes/pybind11. It's opaque to torch.compile's Inductor compiler.
2. `nn.Linear` modules for Q/K/V are separate Python objects with independent `forward()` calls. torch.compile traces at the Python operator level, not the PTX level.
3. The Q/K/V projection matmuls produce tensors that must exist in HBM because FlashAttention reads them from HBM.

**NSL has none of these limitations.** The compiler sees the entire transformer block as an AST. It generates all PTX itself. There are no pre-compiled library boundaries.

---

## 2. CSHA: Three Levels of Fusion

CSHA operates at three levels, each requiring deeper compiler analysis than the last.

### 2.1 Level 1: Boundary Fusion (Epilogue/Prologue Absorption)

**What it does:** Absorbs the memory-bound operations at attention's boundaries into the adjacent compute-bound matmul kernels.

**How it works:**

The Q projection matmul `Q = x_norm @ Wq` writes its output to HBM. RoPE then reads Q from HBM, rotates it, and writes it back. This is two HBM accesses for Q that could be zero.

CSHA fuses RoPE into the Q projection's **epilogue**: the kernel that writes matmul results to memory is extended to also apply the rotation before writing. The Q data is rotated in-register immediately after the matmul accumulation, before any HBM store.

Similarly, RMSNorm is absorbed as a **prologue** of the Q/K/V projections: instead of normalizing `x` in a separate kernel and writing `x_norm` to HBM, the projection matmul reads `x` and applies the normalization on-the-fly as it loads tiles.

**Fusion pattern:**

```
BEFORE (separate kernels):
  x_norm = rmsnorm_kernel(x)        ← read x, write x_norm
  Q = matmul_kernel(x_norm, Wq)     ← read x_norm, write Q
  Q' = rope_kernel(Q, cos, sin)     ← read Q, write Q'

AFTER (fused):
  Q' = matmul_with_norm_rope_kernel(x, Wq, weight, cos, sin)
       ← read x once, write Q' once. x_norm and Q never touch HBM.
```

**HBM traffic reduction:** For Q projection alone, this eliminates 4MB of reads and 4MB of writes. Across Q/K/V projections: **~16MB eliminated per attention sublayer**.

**NSL implementation:** This uses the existing epilogue fusion infrastructure (M31). The compiler detects the pattern `rmsnorm → matmul → rope` in the AST and rewrites it as a fused kernel. The fused PTX is generated at compile time with the RoPE rotation constants baked in.

### 2.2 Level 2: Projection-Attention Pipelining

**What it does:** Overlaps Q/K/V projection computation with FlashAttention consumption, eliminating the need to fully materialize Q/K/V in HBM.

**How it works:**

Instead of computing all of Q, then all of K, then all of V, and *then* starting FlashAttention, CSHA pipelines the computation: Q/K/V projections produce tiles that are consumed by FlashAttention as they become available, using shared memory as the communication channel.

This is directly analogous to FA3's producer-consumer warp specialization, but extended *across* the matmul-attention boundary:

```
Producer warps:
  for each q_tile in 0..num_q_tiles:
    Load x[q_tile] from HBM
    Compute Q_tile = RMSNorm(x[q_tile]) @ Wq + RoPE    (in registers)
    Store Q_tile to shared memory ping buffer
    Signal consumers

    for each kv_tile in 0..num_kv_tiles:
      Load x[kv_tile] from HBM (or reuse if kv_tile overlaps q_tile)
      Compute K_tile = RMSNorm(x[kv_tile]) @ Wk + RoPE
      Compute V_tile = RMSNorm(x[kv_tile]) @ Wv
      Store K_tile, V_tile to shared memory
      Signal consumers

Consumer warps:
  (Standard FlashAttention tiled loop, but reading Q/K/V from SMEM
   instead of HBM, and writing O_acc to SMEM for output projection)
```

**The critical insight:** Q, K, V tiles are produced in shared memory and consumed by FlashAttention from shared memory. They never touch HBM. The only HBM accesses are reading `x` (once) and writing the final output (once).

**HBM traffic reduction:** For NSLCoder-50M (D=512, batch=1, seq=1024):
- Before: ~40MB total HBM traffic per attention sublayer
- After: ~8MB (read x + write output only)
- **5× reduction in HBM traffic**

**Why this is hard:** The producer warps need enough shared memory to hold the projection weights AND the Q/K/V tiles simultaneously. For D=512: Wq is 512×512 = 1MB (fp16), which exceeds H100's 228KB SMEM. Solution: tile the weight matrices too, computing partial projections and accumulating.

**NSL implementation:** This requires a new kernel template in `flash_attention.rs` that interleaves projection and attention. The compiler's shape algebra verifies tile compatibility and the cost model determines whether pipelining is profitable (it isn't for very small models where the projection matmuls are too small to pipeline).

### 2.3 Level 3: Full Block Fusion (Attention + FFN Co-scheduling)

**What it does:** Fuses the entire transformer block into a single kernel launch, with attention and FFN as phases within the same kernel.

**How it works:**

After Level 2 produces the attention output in shared memory, the output projection `Out = Attn_out @ Wo` and residual add `x + Out` can be computed without writing `Attn_out` to HBM. The residual result becomes the input to the FFN sublayer's RMSNorm, which can begin immediately.

The FFN gate/up projections can also be pipelined: as attention output tiles complete, FFN begins processing them. The entire block becomes a streaming pipeline:

```
Persistent Kernel (one CTA per token-block):
  Phase 1: Read x from HBM
  Phase 2: RMSNorm → Q/K/V projection → RoPE → FlashAttention (all in SMEM)
  Phase 3: Output projection → Residual add (write intermediate to SMEM, not HBM)
  Phase 4: RMSNorm → SwiGLU FFN → Residual add (in SMEM)
  Phase 5: Write final output to HBM
```

**HBM accesses for entire block:** ONE read of x, ONE write of output. Everything else stays on-chip.

**Constraints:** This requires enormous SMEM (to hold all weight tiles being actively processed) and is only practical for small-to-medium models (D ≤ 1024 on H100). For larger models, Level 2 pipelining is the practical ceiling.

**NSL implementation:** The compiler's memory planner determines whether full block fusion is feasible given the target GPU's SMEM capacity and the model's dimensions. If not feasible, it falls back to Level 2 or Level 1.

---

## 3. Per-Layer Kernel Specialization

Because NSL generates attention PTX at compile time, it can generate a *different* kernel for each layer based on static analysis.

### 3.1 Weight-Informed Sparsity (with `--weights`)

When pre-trained weights are available at compile time (`nsl build --weights model.safetensors`), the compiler analyzes Q/K attention weight matrices to determine per-layer attention characteristics:

**Head importance analysis:** Compute the Frobenius norm of each attention head's Wq and Wk projections. Heads with very small norms contribute little to attention and can be pruned at compile time, generating a kernel that computes fewer heads.

**Attention entropy estimation:** Using the pre-trained Q/K weights, estimate the entropy of attention distributions. Low-entropy layers (focused attention) benefit from sparse kernels. High-entropy layers (diffuse attention) need full attention.

**Compile-time output:**
```
[CSHA] Layer 0: 8/8 heads active, entropy=high → full FA2 kernel
[CSHA] Layer 1: 8/8 heads active, entropy=high → full FA2 kernel
[CSHA] Layer 5: 6/8 heads active (2 pruned, norm<0.01), entropy=medium → 6-head kernel
[CSHA] Layer 7: 8/8 heads active, entropy=low → block-sparse kernel (top-k=64)
```

### 3.2 Roofline-Adapted Tile Sizes

NSL's cost model determines optimal tile sizes per layer based on the actual compute/memory balance:

- **Memory-bound layers** (small batch, long sequence): Larger tiles to improve arithmetic intensity
- **Compute-bound layers** (large batch, short sequence): Smaller tiles to reduce register pressure
- **Layers with pruned heads**: Adjusted grid dimensions to avoid launching empty warps

Currently, FlashAttention uses fixed tile sizes (block_q=128, block_kv=128 for FA2; block_q=64, block_kv=64 for FA3). CSHA can use different sizes per layer.

### 3.3 Per-Head Mixed Precision

The compiler can determine optimal precision per attention head:

- Heads with high-magnitude weights → FP16 (need dynamic range)
- Heads with small, clustered weights → FP8 E4M3 (can tolerate lower precision)
- Value projections → FP8 E5M2 (wider range for accumulation)

This is generated at compile time as a single kernel with mixed-precision MMA instructions for different heads.

---

## 4. Compiler-Determined Attention Patterns

### 4.1 Static Causal Mask Optimization

NSL's shape algebra knows the sequence length at compile time when using bounded dimensions (`SeqLen < 4096`). For short sequences, the causal mask wastes significant compute — half the QK^T entries are masked. The compiler can:

- For seq_len ≤ block_kv: Skip the entire masking branch (single tile, no mask needed)
- For seq_len with known alignment: Generate optimized mask that uses bitwise ops instead of comparisons
- For bounded seq_len: Pre-compute the number of valid tiles and embed it as a compile-time constant

### 4.2 GQA Compile-Time Expansion

NSL's GQA implementation currently expands KV heads at runtime:

```nsl
let k5 = k_rope.reshape([batch, nkv, 1, seq_len, hd])
let k_exp = k5.expand([batch, nkv, n_rep, seq_len, hd]).contiguous().reshape([batch, nh, seq_len, hd])
```

This requires `contiguous()` which copies data. CSHA eliminates this by generating a kernel where the KV head replication is expressed as a stride pattern in the shared memory addressing — the same KV data is read multiple times with different thread-to-output mappings. Zero-copy GQA expansion.

### 4.3 Compile-Time Attention Sink Detection

For autoregressive models, the first few tokens (attention sinks) consistently receive high attention regardless of query position. The compiler can detect this from weight analysis and generate a kernel with a specialized "sink cache" that keeps the first N tokens permanently in shared memory or registers, avoiding repeated HBM loads.

---

## 5. Source-AD Optimized Backward Pass

NSL's source AD generates attention backward passes that are aware of the specific forward configuration. Unlike the generic FlashAttention backward, CSHA's backward:

### 5.1 Fused Backward with Projection Gradients

The standard backward computes dQ, dK, dV from the attention backward, then separately computes gradients for Wq, Wk, Wv, Wo. CSHA fuses these:

```
Standard (4 separate kernel launches):
  1. FA backward: dO → dQ, dK, dV (from saved Q, K, V, O, logsumexp)
  2. dWq = x_norm^T @ dQ
  3. dWk = x_norm^T @ dK
  4. dWv = x_norm^T @ dV

CSHA backward (1-2 kernel launches):
  1. Fused: dO → dQ, dK, dV, dWq, dWk, dWv, dWo, dx
     (accumulate weight gradients as attention backward produces dQ/dK/dV tiles)
```

### 5.2 Dead Head Gradient Elimination

If the compiler determined that certain heads were pruned in the forward pass (Section 3.1), the backward pass skips gradient computation for those heads entirely. This isn't just zeroing the gradient — the backward kernel literally doesn't contain the code for pruned heads.

### 5.3 Checkpointing-Aware Backward

CSHA's backward is generated by the source AD system, which can apply `@checkpoint` at tile granularity rather than layer granularity. Instead of saving all of Q, K, V for the backward (standard FA), or recomputing the entire forward (standard checkpointing), CSHA can recompute individual Q/K/V tiles on-demand during the backward, using the fused norm+projection+RoPE from Level 1.

---

## 6. NSL Language Surface

### 6.1 Transparent Usage

CSHA requires no syntax changes. The existing `scaled_dot_product_attention` intrinsic and `GroupedQueryAttention` model are automatically optimized:

```nsl
# This model's forward pass automatically gets CSHA optimization
model TransformerBlock(d_model: int, n_heads: int, n_kv_heads: int):
    attn_norm: RMSNorm = RMSNorm(d_model)
    attn: GroupedQueryAttention = GroupedQueryAttention(d_model, n_heads, n_kv_heads, 0.1)
    ffn_norm: RMSNorm = RMSNorm(d_model)
    ffn: SwiGLUFFN = SwiGLUFFN(d_model, d_ff, 0.1)

    fn forward(self, x: Tensor, training: bool) -> Tensor:
        # Compiler detects: norm → GQA(contains proj + rope + sdpa) → residual
        # Automatically generates CSHA Level 1-2 kernel
        let h = x + self.attn.forward(self.attn_norm.forward(x), training)
        return h + self.ffn.forward(self.ffn_norm.forward(h), training)
```

### 6.2 Explicit Control

```nsl
# Force specific CSHA level
@csha(level=2, target=h100)
model TransformerBlock(d_model: int, n_heads: int):
    ...

# Disable CSHA for debugging
@csha(disable=true)
```

model TransformerBlock(d_model: int, n_heads: int):
    ...
```

### 6.3 Compile-Time Diagnostics

```bash
$ nsl check --csha-report model.nsl --weights model.safetensors --target h100

=== CSHA Compilation Report ===
Target: NVIDIA H100-SXM (228KB SMEM, 989 TFLOPS FP16, 3.35 TB/s HBM)
Model: NSLCoder-50M (D=512, H=8, KV=4, 8 layers)

Layer 0: CSHA Level 2 (projection-attention pipelining)
  Tile config: block_q=64, block_kv=64, head_dim=64
  SMEM usage: 147KB / 228KB (64%)
  Fusions: RMSNorm→Wq+RoPE, RMSNorm→Wk+RoPE, RMSNorm→Wv
  GQA: zero-copy expansion (stride pattern)
  Precision: Q=FP16, K=FP16, V=FP16, Acc=FP32
  HBM traffic: 8.2MB (vs 42MB unfused) — 5.1× reduction
  Est. time: 2.8μs (vs 6.9μs unfused) — 2.5× speedup

Layer 7: CSHA Level 2 + sparse
  2/8 heads pruned (norm < 0.01)
  Block-sparse: top-k=128 per query (entropy=0.31)
  HBM traffic: 5.1MB — 8.2× reduction
  Est. time: 1.9μs — 3.6× speedup

Block-level fusion: Level 3 NOT feasible (Wq+Wk+Wv+Wo = 2MB > 228KB SMEM)
  → Using Level 2 with epilogue-fused output projection

Backward pass: Source-AD fused backward
  Gradient fusions: dWq/dWk/dWv accumulated during attention backward
  Dead head elimination: 2 heads skipped in layers 5-7
  Checkpoint strategy: tile-granular recompute for norm+proj+RoPE
```

---

## 7. Performance Projections
```

### 7.1 NSLCoder-50M (D=512, 8 layers, batch=1, seq=1024)

| Configuration | HBM Traffic/Layer | Time/Layer | Forward (8 layers) |
|---|---|---|---|
| Naive attention (no FA) | ~134MB | ~45μs | 360μs |
| FlashAttention-2 | ~42MB | ~6.9μs | 55μs |
| CSHA Level 1 (boundary fusion) | ~24MB | ~4.1μs | 33μs |
| CSHA Level 2 (pipelining) | ~8MB | ~2.8μs | **22μs** |

### 7.2 Scaling Analysis

CSHA's benefit scales with model dimensions and hardware characteristics:

- **Larger models (D=4096):** Projections become more compute-bound, so boundary fusion (Level 1) provides less relative savings. However, the absolute HBM traffic eliminated is larger.
- **Longer sequences:** FlashAttention's internal optimization becomes more important ($O(S^2)$ compute). CSHA's boundary savings are $O(S \cdot D)$, which becomes proportionally smaller. But Level 2 pipelining still eliminates Q/K/V materialization.
- **Higher batch sizes:** All operations become more compute-bound. CSHA Level 1 provides diminishing returns. Level 2 remains valuable for eliminating Q/K/V HBM traffic.
- **Smaller models ($D \leq 1024$):** CSHA Level 3 (full block fusion) becomes feasible, providing the maximum benefit.

---

## 8. Comparison with Related Work

| System | Fuses Inside Attention | Fuses Across Projection Boundary | Fuses Norm/RoPE | Per-Layer Specialization | Compile-Time Sparsity |
|---|---|---|---|---|---|
| FlashAttention-2 | Yes (QK^T→softmax→PV) | No | No | No | No |
| FlashAttention-3 | Yes + warp specialization | No | No | No | No |
| torch.compile | Elementwise only | No | Partial (norm) | No | No |
| DeepFusionKernel | No (FFN only) | No | No | No | No |
| Triton + custom | Partial | Manual only | Manual only | Manual | No |
| **CSHA (NSL)** | **Yes** | **Yes (Level 2)** | **Yes (Level 1)** | **Yes (auto)** | **Yes (weight-informed)** |

The fundamental difference: every other system treats FlashAttention as a black box. CSHA treats the *entire attention computation* as compiler-analyzable source code, because in NSL, it is.

---

## 9. Implementation Plan

### 9.1 Level 1 (Boundary Fusion) — Low Risk, High Value

Extends existing M31 epilogue fusion to recognize the `rmsnorm → matmul → rope` pattern. Estimated ~800 LOC in `nsl-codegen`, mostly in a new `csha_boundary.rs` file that generates fused PTX templates.

**Dependencies:** epilogue_fusion.rs, backend_ptx.rs, flash_attention.rs

### 9.2 Level 2 (Pipelining) — Medium Risk, Very High Value

New kernel template that interleaves projection computation with FlashAttention's KV-tile loop. This is the largest implementation effort (~2000 LOC) and requires careful shared memory layout planning.

**Dependencies:** flash_attention.rs (major modifications), cost_model.rs, memory_planner.rs

### 9.3 Weight-Informed Specialization — Low Risk, Medium Value

Extends M52 weight-aware compilation to analyze attention heads. ~600 LOC for head pruning analysis and per-layer kernel config selection.

**Dependencies:** weight_aware.rs, flash_attention.rs

### 9.4 Source-AD Fused Backward — Medium Risk, High Value

Extends M40 source AD to generate fused attention backward passes. ~1200 LOC.

**Dependencies:** source_ad.rs, wengert.rs, ad_rules.rs

### 9.5 Integration with Existing NSL Features

CSHA composes with all existing attention features:

| Feature | Integration |
|---|---|
| Paged KV (M25) | Level 2 producers load from page table instead of contiguous K/V |
| Speculative decoding (M33) | Tree mask baked into CSHA kernel via compile-time config |
| FP8 (M35) | Per-head precision selection integrated into CSHA kernel |
| Context parallel (M34) | Ring communication fused into producer warps |
| Memory planning (M36) | SMEM layout optimized by planner for CSHA's multi-phase usage |

---

## 10. Conclusion

CSHA represents a fundamental rethinking of attention optimization. Where FlashAttention asks *"how do we compute attention faster?"*, CSHA asks *"why are we computing attention as a separate operation at all?"*

The answer — that attention is separated from its surrounding operations because framework boundaries (Python modules, pre-compiled CUDA libraries, dynamic dispatch) prevent fusion — is a *software engineering* answer, not a *mathematical* one. There is no mathematical reason that RMSNorm, Q/K/V projection, RoPE, and tiled attention must be separate kernels. They are separate because PyTorch's architecture makes them separate.

NSL has no such boundaries. The compiler sees the full transformer block as an AST, generates all PTX itself, and can fuse any computation that the hardware supports fusing. CSHA exploits this to eliminate the HBM round-trips between operations that should never have been separate — producing attention sublayers that are 2-5× faster than FlashAttention-3 on the same hardware.

---

## References


1. Dao, T. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." NeurIPS 2022.
2. Dao, T. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." ICLR 2024.
3. Shah, J., et al. "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision." NeurIPS 2024.
4. Yuan, J., et al. "Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention." arXiv 2502.11089, 2025.
5. Williams, S., Waterman, A., Patterson, D. "Roofline: An Insightful Visual Performance Model." CACM 2009.
6. Ainslie, J., et al. "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints." EMNLP 2023.
7. Su, J., et al. "RoFormer: Enhanced Transformer with Rotary Position Embedding." Neurocomputing 2024.